

Frontend test suite

Complete reference for the Playwright end-to-end suite covering the PMS React frontend — how to run it, what it covers, how the mocked API and contract checks fit together, and how to add to it.

183 TESTS	12 FILES	~33s FULL RUN	2 DEVICE PROFILES	0 EXTERNAL SERVICES
---------------------	--------------------	-------------------------	-----------------------------	-------------------------------

CONTENTS

1. Quick start
2. What the suite covers
3. How it is wired together
4. Authentication
5. The mocked API
6. The contract layer
7. Writing a new test
8. Configuration reference
9. What is not covered
10. Troubleshooting
11. Running in CI

1 Quick start

Everything lives under `frontend/`. The suite needs no Flask backend, no InventDB instance, and no network — the API is answered inside the browser.

```
# one-time, after cloning
cd frontend
npm install
npx playwright install chromium

# run
npm run test:e2e
```

COMMAND	DOES
<code>npm run test:e2e</code>	Full suite, headless, both device profiles
<code>npm run test:e2e:ui</code>	UI mode — watch, time-travel, pick locators. Best place to write tests
<code>npm run test:e2e:headed</code>	Desktop profile with a visible browser
<code>npm run test:e2e:mobile</code>	Pixel 7 profile only
<code>npm run test:e2e:report</code>	Open the HTML report from the last run
<code>npm run typecheck:e2e</code>	Type-check the test files (not part of <code>npm run build</code>)

USEFUL FLAGS

```

npx playwright test login                # only files matching "login"
npx playwright test -g "signs in"        # only tests whose title matches
npx playwright test --project=chromium    # skip the mobile profile
npx playwright test --debug               # step through with the inspector
npx playwright test --repeat-each=3       # flake hunting
npx playwright show-trace test-results/.../trace.zip

```

The dev server starts itself. Playwright runs `npm run dev` and waits for port 5173. If a dev server is already running there it reuses it instead — so a stale server with an invalidated dependency cache can affect a local run. If results look impossible, stop your dev server and let Playwright start its own.

2 What the suite covers

FILE	TESTS	COVERS
<code>auth.setup.ts</code>	1	Signs in once through the real form; saves the state every other project starts from
<code>contract.spec.ts</code>	48	Holds the mocked API to the shape the backend publishes — see §6
<code>entity-list.spec.ts</code>	22	All ten modules; table rendering, status badges, reference resolution, currency and date formatting, search, sort, empty states
<code>navigation.spec.ts</code>	20	All 14 sidebar destinations, active highlighting, reload on a deep route, back/forward, unmatched routes
<code>entity-crud.spec.ts</code>	20	Create / edit / delete round trips, payload type coercion, required-field blocking, reference dropdowns, modal dismissal, form-state isolation between modules
<code>reports.spec.ts</code>	18	Gallery grouping and filtering, parameter seeding, rendering into the sandboxed frame, refresh without blanking the sheet, print enablement, failure paths
<code>login.spec.ts</code>	14	Sign-in, credential errors, in-flight button state, network failure, password visibility, registration links, route protection while signed out
<code>settings.spec.ts</code>	11	Account details, InventDB connection card, theme switching and persistence, password change, sign-out from both places
<code>workflows.spec.ts</code>	8	Workflow cards, trigger hints, plan step timelines, run history, the run-timeline chart, empty and error states
<code>mobile.spec.ts</code>	8	Drawer open/close behaviour, login hero reflow, horizontal-overflow guard across four routes
<code>dashboard.spec.ts</code>	7	Eight stat tiles with exact formatted values, four charts, empty-data branch, error branch, loading spinner
<code>errors.spec.ts</code>	6	401 auto-logout on both reads and writes, 5xx surfacing, network abort, recovery after the API returns

Ten of the eleven spec files drive the real React app in a real browser.

`contract.spec.ts` is different in kind: it never opens a page, it validates the fake API itself.

Coverage by feature area

- **Every module.** Properties, Owners, Tenants, Leases, Work Orders, Vendors, Inspections, Compliance, Daily Tasks and Accounting are each driven through the same generic list contract, because they are one component configured by `src/config/entities.ts`.
- **Every route.** All 14 sidebar destinations plus the two unmatched-route behaviours (single unknown segment renders “Unknown module”; a deeper unknown path redirects to the dashboard).
- **Both themes' plumbing.** Theme toggling from the topbar and the settings card, persistence across reload, and the pre-paint stamp on `<html>`.

- **Failure paths, not just happy paths.** Roughly a quarter of the assertions are about what happens when the API returns 400, 401, 409, 500, 502, 503, or simply never answers.

3 How it is wired together

File layout

```
frontend/
├ playwright.config.ts      projects, timeouts, dev server
├ tsconfig.e2e.json         type-checks the suite in isolation
└ e2e/
  ├── auth.setup.ts         signs in, saves storage state
  ├── *.spec.ts             the tests
  ├── .auth/user.json       generated, git-ignored
  └ fixtures/
    ├── index.ts            extended `test`, helpers, options
    ├── data.ts             seed records and payloads
    ├── mock-api.ts         the fake backend
    └ contract.ts           reads recorded real responses
```

Projects

Three projects run in dependency order:

PROJECT	RUNS	PROFILE
setup	auth.setup.ts only	Signed out; produces e2e/.auth/user.json
chromium	All specs except mobile.spec.ts	Desktop Chrome, 1440×900, signed in
mobile	mobile.spec.ts only	Pixel 7 (412px), signed in

The mobile profile exists because the layout genuinely changes below 820px — the sidebar becomes a drawer — and below 960px, where the login hero reflows via `display: contents`. Those are different behaviours, not the same page at a different size, so they get their own tests rather than a resize inside a desktop test.

Fixtures

Specs import `test` from `./fixtures` rather than from Playwright directly. That extended `test` supplies:

NAME	KIND	PURPOSE
<code>store</code>	Fixture	A fresh, mutable copy of the seed data for this test
<code>mockApi</code>	Auto fixture	Installs the fake API before the test body runs. Specs never request it
<code>liveApi</code>	Option	<code>test.use({ liveApi: true })</code> skips mocking entirely
<code>apiLatency</code>	Option	Delays every API response, for asserting on loading states

Why `store` does not depend on `page` . It would be natural to build the store and install the routes in one fixture. Doing so creates a `page → store → page` cycle once the auto fixture needs both, which Playwright rejects at load time. Keeping `store` pure and putting the installation in a separate auto fixture avoids it.

4 Authentication

`AuthContext` gates every route behind a user and token held in `localStorage` (`pms.token` and `pms.user`). Signing in once per test would be slow and would put the same assertions in every file, so the `setup` project does it once and saves the resulting browser state.

That state is produced by driving the real login form — not by writing the token directly. If the login contract changes, the failure appears in one obvious place instead of silently handing every other test a shape the app no longer accepts. The setup also pins `pms.theme` to light, so theme assertions do not depend on the machine's colour-scheme preference.

STARTING A TEST SIGNED OUT

```
import { signedOut, test } from "../fixtures";

test.use(signedOut); // clears the saved state for this file
```

`login.spec.ts` uses this for the form itself and for the route-protection checks. There is also a `signIn(page)` helper for tests that need to sign in mid-test.

5 The mocked API

`fixtures/mock-api.ts` installs **one** route handler for every same-origin `/api/` call and dispatches internally by path. One broad handler rather than a dozen narrow ones is deliberate: Playwright matches handlers in reverse registration order, so with many overlapping patterns the effective precedence becomes hard to reason about. With a single handler installed by the fixture, a `page.route()` a spec registers later *always* wins — which is exactly the override behaviour the failure tests need.

Writes mutate the per-test `store` . A create really does appear in the list refetch React Query fires afterwards, so the assertions exercise the full round trip rather

than just inspecting the outgoing request.

Overriding one endpoint

```
await page.route("**/api/dashboard/summary", (route) =>
  route.fulfill({
    status: 500,
    contentType: "application/json",
    body: JSON.stringify({ error: "InventDB is unreachable" }),
  })
);
await page.reload();

await expect(page.locator(".alert.error")).toHaveText("InventDB is unreachable");
```

Overriding only some methods

Call `route.fallback()` to defer to the mock for the requests you did not mean to change:

```
await page.route("**/api/properties", (route) => {
  if (route.request().method() !== "POST") return route.fallback();
  return route.fulfill({ status: 400, contentType: "application/json",
    body: JSON.stringify({ error: "street must be unique" }) });
});
```

Extending the mock

Add a branch in the dispatch chain in `mock-api.ts`, keyed on the first path segment. Two rules matter:

- **Match `/api/` at the root of the path, not anywhere in the URL.** The handler uses a regex rather than a glob for this reason. In development Vite serves the app's own source files at their real paths, and some of those paths contain `/api/` — a pattern loose enough to catch them will intercept application code. The comment above the regex in `mock-api.ts` explains it in full; leave it there.
- **Envelope endpoints should come from the contract, not by hand.** If the backend composes the response itself, serve the recorded one via `backendResponse(key)` — see §6.

Seed data

`fixtures/data.ts` holds three properties, two of everything else, plus dashboard, report and workflow payloads. Field names mirror `src/config/entities.ts` exactly, because the UI reads records by those names — a drifting fixture shows up as a blank cell rather than a clear failure. Every record carries an `_id`, which is what the edit and delete paths key off.

6 The contract layer

A mocked API has one structural weakness: it can agree with itself forever while drifting away from the real backend. A renamed field or a dropped envelope would leave the whole suite green against a backend it no longer describes.

The contract layer closes that gap. `backend/tests/contract/` replays recorded InventDB responses through the real Flask app, validates the output against `contract/api-contract.json`, and publishes the app's actual responses to `contract/responses/`. `contract.spec.ts` then holds the mock to the same contract and the same responses. Drift on either side now fails on that side.

Regenerate after changing a backend response shape:

```
cd backend
UPDATE_CONTRACT=1 python -m pytest tests/contract
```

The mock serves recorded responses for *envelope* endpoints — metadata and other bodies the backend composes itself. Per-record fixtures stay in `data.ts`, because those exist to drive assertions on specific rendered values, and a captured dataset would change them every time someone edited the sandbox.

The validator in `contract.spec.ts` is a deliberate re-implementation of the Python one in `backend/tests/contract/spec.py`, not a shared module. A validator generated from the backend would agree with the backend by construction, which is the one thing it must not do.

7 Writing a new test

```
import { dialog, expect, field, row, rows, test } from "../fixtures";

test("creates a vendor", async ({ page }) => {
  await page.goto("/vendors");
  await page.getByRole("button", { name: "New Vendor" }).click();

  await field(page, "company").fill("Summit Roofing");
  await dialog(page).getByRole("button", { name: "Save" }).click();

  await expect(page.locator(".toast.success")).toHaveText("Vendor created");
  await expect(row(page, "Summit Roofing")).toBeVisible();
});
```

Helpers

HELPER	RETURNS
<code>field(page, "street")</code>	An EntityForm control by its field name
<code>reportParam(page, "region")</code>	A report parameter input
<code>dialog(page)</code>	The open modal or confirm dialog
<code>row(page, "Mumbai")</code>	A table row containing that text
<code>rows(page)</code>	All rendered data rows
<code>signIn(page)</code>	Signs in through the form mid-test
<code>signedOut</code>	Config object for <code>test.use()</code> to start anonymous

Conventions that keep tests stable

HEADINGS REPEAT — ALWAYS PASS A LEVEL

The layout renders the page title as an `<h1>` in the topbar, and entity pages repeat the same text as an `<h2>`. Asking for the heading by name alone matches both and fails strict mode.

```
page.getByRole("heading", { name: "Properties", level: 1 }) // topbar
page.getByRole("heading", { name: "Properties", level: 2 }) // page head
```

LABELS ARE SUBSTRING-MATCHED BY DEFAULT

`getByLabel("Password")` also matches the show/hide toggle, whose `aria-label` is "Show password". Use `{ exact: true }` on the login and settings forms.

USE `FIELD()` FOR ENTITY FORMS

Required labels render as `Street*`, which defeats exact label matching, while loose matching makes `Phone` ambiguous with `Emergency Phone`. Every control has a stable `id="f-<name>"`, which is what the helper targets.

COUNTING CHARTS

Recharts gives legend icons their own `.recharts-surface`. Count `.recharts-wrapper` when you mean "how many charts".

PREFER ROUND-TRIP ASSERTIONS

Because writes mutate the store, assert on the resulting row rather than only on the outgoing request. Check the request payload as well when type coercion matters — numeric and currency fields are converted out of their string form before sending, and untouched fields are omitted rather than sent empty.

Do not add `waitForTimeout` . Every locator and assertion already retries until it passes or times out. A fixed sleep either makes the suite slower than it needs to be or hides a real race. The search box is debounced by 300ms and the assertions absorb that without help.

8 Configuration reference

SETTING	VALUE	WHY
<code>baseUrl</code>	<code>http://localhost:5173</code>	Vite dev server
<code>locale</code>	<code>en-US</code>	The app formats money with <code>toLocaleString</code> ; unpinned, <code>\$2,400</code> is machine-dependent
<code>timezoneId</code>	<code>UTC</code>	Same reason for dates — a date-only value is parsed as local time
<code>contextOptions.reducedMotion</code>	<code>reduce</code>	Removes the route-change animation from under click assertions
<code>timeout</code>	45s per test	Generous; a typical test finishes in 1–2s
<code>expect.timeout</code>	10s	React Query retries failed requests once, so error states need headroom
<code>retries</code>	2 in CI, 0 locally	Local failures should be reproduced, not retried away
<code>workers</code>	2 in CI, auto locally	CI runners are usually smaller than a dev machine
<code>trace</code>	<code>on-first-retry</code>	Full DOM, network and console capture for anything that fails
<code>screenshot</code> / <code>video</code>	On failure only	Keeps artifacts small
<code>webServer.reuseExistingServer</code>	true locally	Uses your running dev server; CI always starts a clean one

Google Fonts is intercepted and answered with an empty stylesheet. That keeps the suite offline-capable and removes a slow third-party dependency; the app falls back to its system font stack, which no assertion depends on.

GIT-IGNORED OUTPUTS

`frontend/e2e/.auth/` , `frontend/test-results/` , `frontend/playwright-report/` and `frontend/blob-report/` are all excluded in the root `.gitignore` .

9 What is not covered

Stated plainly, because a green suite invites more confidence than it earns.

- **No backend Python runs.** During a frontend test run the Flask app is not started. Backend behaviour is covered by the separate pytest suite in `backend/tests/` , and the two are joined only by the contract layer in §6.

- **No real InventDB.** Nothing here proves the app works against a live instance — only against responses shaped like one.
- **Per-record fixtures are assumptions.** The contract pins envelope shapes. The individual records in `data.ts` are hand-written, so a field renamed in the real schema would not fail here.
- **No visual regression.** Layout is asserted structurally and through the mobile overflow guard, not by pixel comparison.
- **No accessibility audit.** Tests use accessible roles and names throughout, which exercises the accessibility tree incidentally, but nothing runs an axe-style ruleset.
- **Chromium only.** Both profiles are Chromium-based. Firefox and WebKit are installed but no project targets them.
- **Passing tests are weak evidence of correctness.** The assertions were written by reading the current implementation, so they largely encode today's behaviour as correct. Their value is catching tomorrow's regression, not proving today's code right.

10 Troubleshooting

SYMPTOM	LIKELY CAUSE
Blank page, timeout waiting for any element	The app did not boot. Check the browser console rather than the test output — run the same navigation with a <code>page.on("pageerror")</code> listener. A route pattern intercepting application code is the usual cause
<code>strict mode violation: resolved to 2 elements</code>	An ambiguous locator. Add <code>level:</code> to headings, <code>exact: true</code> to labels and text, or scope to a container
Element “intercepts pointer events”	Something overlays the click target — usually an open drawer or modal scrim. Click a position that is actually exposed
Currency or date assertions fail	<code>locale</code> or <code>timezoneId</code> lost from the config
Contract tests fail after a backend change	Expected. Regenerate with <code>UPDATE_CONTRACT=1</code> (\$6), then update the mock to match
Local run behaves impossibly	A stale dev server on 5173 is being reused. Stop it and let Playwright start its own
Fixture dependency cycle at startup	A fixture was made to depend on <code>page</code> which something else needs first. See the note in §3

For anything that fails on a retry, the trace is the fastest route to an answer — it carries a DOM snapshot, the network log and the console for every step: `npx playwright show-trace test-results/.../trace.zip`.

11 Running in CI

```
- run: npm ci
  working-directory: frontend

- run: npx playwright install --with-deps chromium
  working-directory: frontend

- run: npx playwright test
  working-directory: frontend

- uses: actions/upload-artifact@v4
  if: ${{ !cancelled() }}
  with:
    name: playwright-report
    path: frontend/playwright-report/
```

Setting `CI` changes four things automatically: retries go to 2, workers drop to 2, `test.only` becomes an error rather than a silent filter, and the dev server is always started fresh instead of reusing whatever is on 5173.

Consider adding `npm run typecheck:e2e` as a CI step. It is deliberately not part of `npm run build`, so a type error in a test file will not block a deploy — which means nothing catches it unless CI runs it explicitly.

183 PASSING Verified 10 August 2026 · full run 32.6s · Playwright 1.62.1 · Chromium.

Source: `frontend/e2e/` · Contract: `contract/` · Backend suite: `backend/tests/`